



Bluetooth RC Car Using Arduino UNO R4

Project Title & Mission

Goal

Design and implement a wireless remote-controlled vehicle.

Core Platform

Arduino UNO R4 WiFi.

Primary Features

Bluetooth Low Energy (BLE) control, mobile app integration, and dual-motor propulsion.

Project Overview

Wireless Control:

Leverages BLE for low-latency communication with mobile devices.

Movement:

Driven by two DC motors via an L298N dual H-bridge motor driver.

Features:

- Adjustable motor speed.
- Active safety using ultrasonic sensors.
- Visual feedback via an integrated LED matrix.

Hardware Components

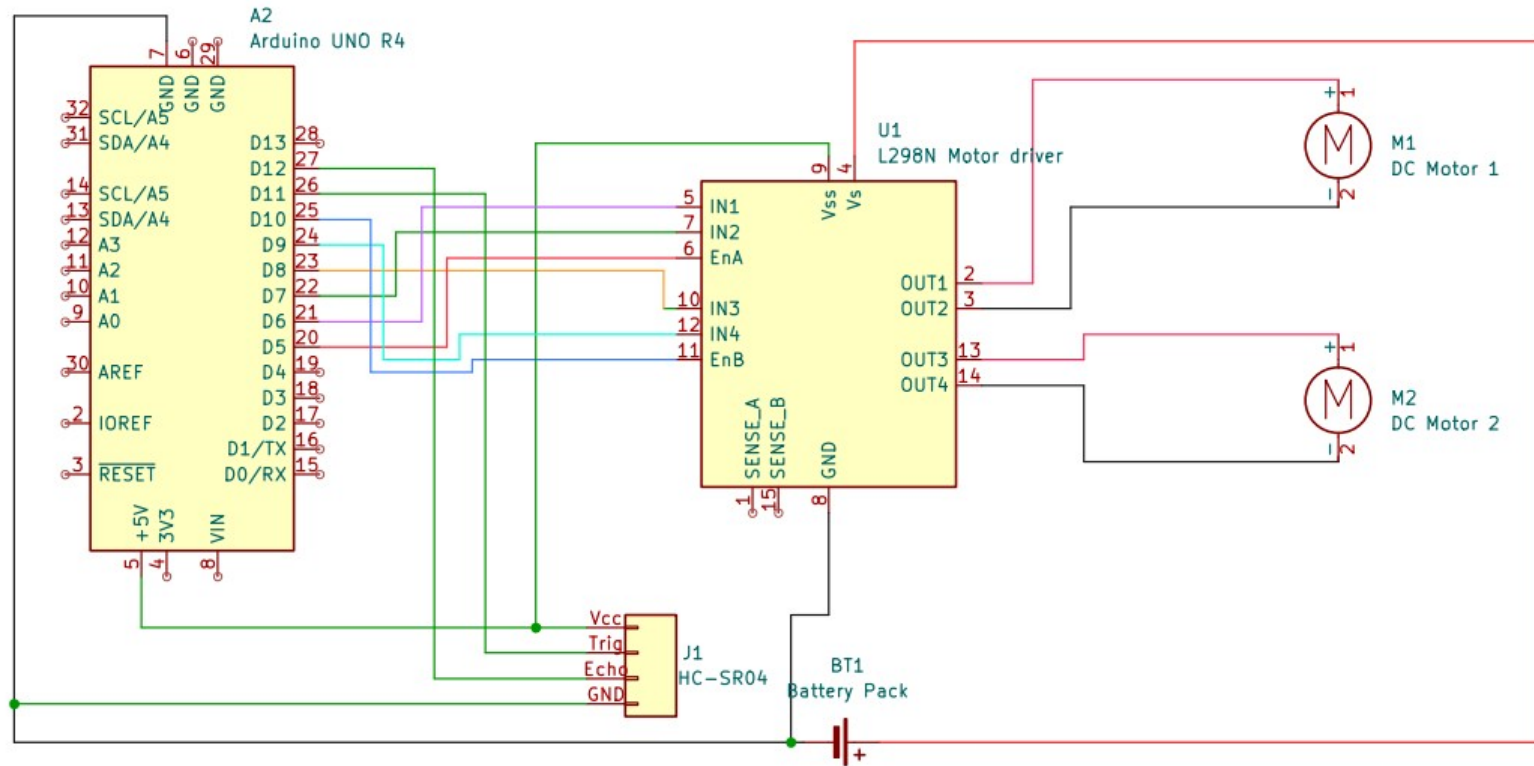
Microcontroller: Arduino UNO R4 WiFi, featuring an ESP32-S3-Mini-1 co-processor for connectivity.

Motor Driver: L298N Dual H-Bridge for handling high-current DC motors.

Sensing: HC-SR04 Ultrasonic Sensor for obstacle detection.

Power: 6 AA battery pack.

Schematic



Software Architecture

Environment

Developed in C++ using the Arduino IDE.

Modular Design

Software is split into independent files for improved readability and extensibility.

Key Modules

- **Main.ino**: Orchestrates the system loop.
- **motor_control**: Logic for motor movement.
- **ble_control**: Handles Bluetooth communication.
- **sensor_control**: Manages ultrasonic data

Motor Control & Logic

Abstraction

The system uses high-level "direction" functions rather than direct pin manipulation.

Speed Control

Managed via analogWrite on ENA and ENB pins

Motion truth table

Direction	IN1	IN2	IN3	IN4	Logic
Forward	HIGH	LOW	HIGH	LOW	Both motors spin forward.
Backward	LOW	HIGH	LOW	HIGH	Polarity is reversed for both motors.
Left Turn	HIGH	LOW	LOW	HIGH	Right motor forward, Left motor backward.
Right Turn	LOW	HIGH	HIGH	LOW	Left motor forward, Right motor backward.
Stop	LOW	LOW	LOW	LOW	All coils de-energized.

BLE Communication

Architecture

Arduino acts as the Peripheral (Server), while the phone acts as the Central (Client).

Service & Characteristic

- **Service:** "CarControlService".
- **Characteristic:** "CommandCharacteristic" receives single-character commands (e.g., 'F', 'B', 'S').
- **Library:** Uses the official ArduinoBLE library.

Mobile App Interface

Application

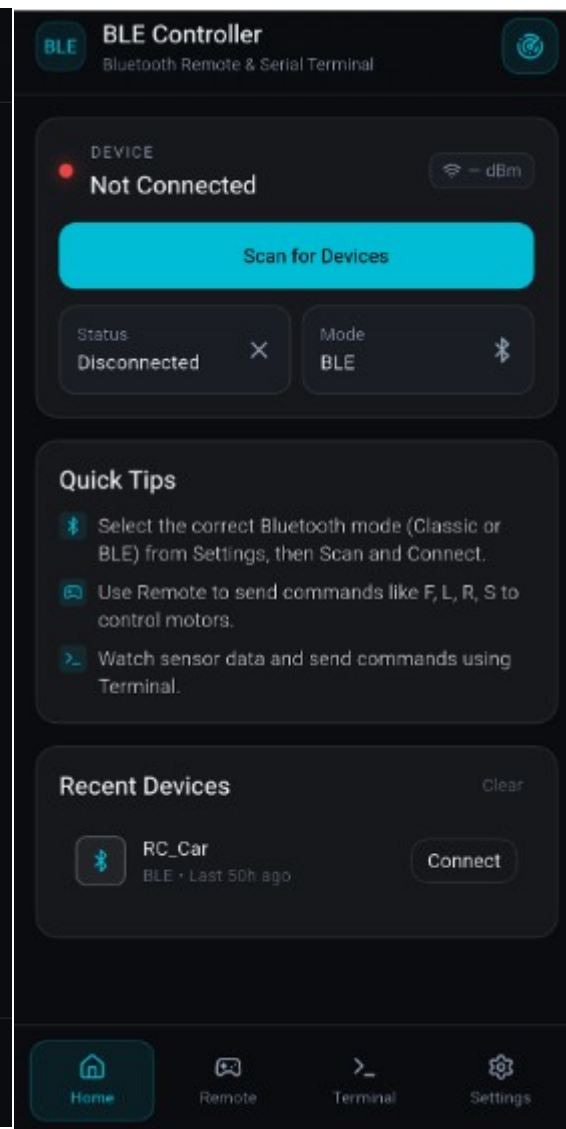
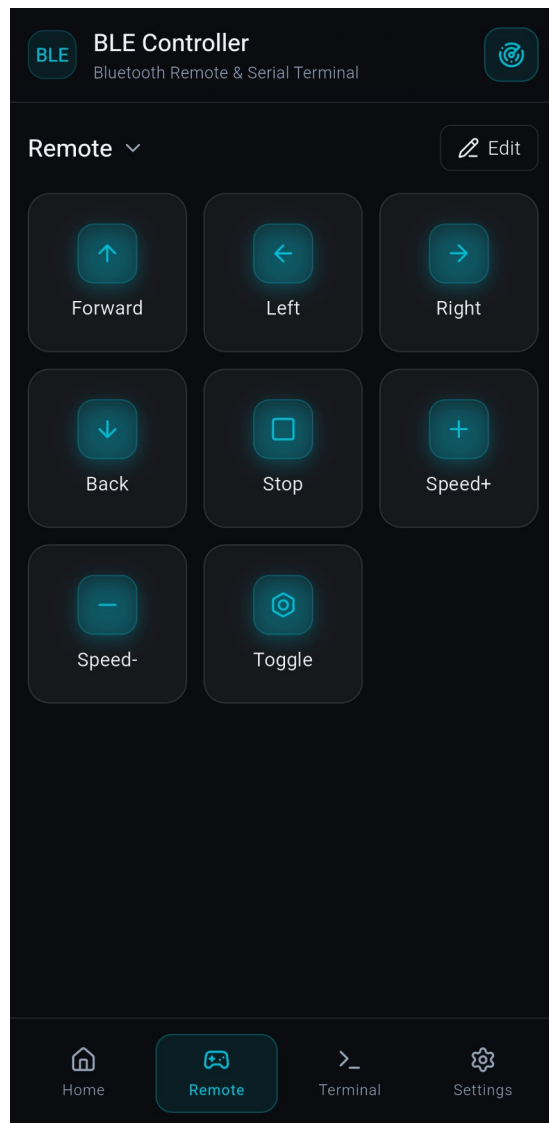
BLE Controller - Arduino
ESP32 by CircuitMagic Inc.

Functionality

Allows the user to create custom buttons and map them to BLE characteristics.

Command Mapping

- **'F' / 'B'**: Forward and Backward.
- **'L' / 'R'**: Left and Right turns.
- **'+' / '-'**: Speed adjustment.
- **'T'**: Toggles between driving modes.



Active Safety & AEB

Collision Prevention:

Uses "Active Sonar" to monitor surroundings.

Automatic Emergency Braking (AEB):

If an obstacle is detected within 15 cm, the car stops immediately and overrides user commands.

Automatic Slowdown:

Speed is automatically capped if an obstacle is detected within 40 cm.

Dual Driving Modes

One-Press Mode (Default)

A single command triggers persistent movement until a "Stop" command or new direction is received.

Hold-to-Move

- Simulates a "dead-man's switch".
- Requires a constant stream of BLE commands; if no command is received within 300ms, the car stops automatically.

Safety Sync

Motors stop immediately when switching modes via the 'T' command.

Visual Feedback

Display

Uses the built-in 12x8 LED matrix on the UNO R4.

Indications

- **Arrows:** Show direction of movement.
- **Icons:** Stop symbols and mode indicators.

Technical Implementation

Bitmap data is stored in flash memory and copied to RAM for memory-safe display.